


# Patch Diffing et archéologie — A-EPI-001

## Du commit au root cause

Ted “Tedsig42” Kouhouenou ted.sig42@tutamail.com

*Summary: Module hors-piscine : reconstruire la root cause d’un correctif à partir du commit, du diff source et, si nécessaire, du diff binaire. Mesurer le patch gap, distinguer correction et atténuation d’urgence, et écrire un finding de niveau professionnel.*

*Prerequisites : Solide lecture de C, git, bases d’assembleur x86-64*

*Version: 1.0*

# Contents


**I Instructions** **2**

**II Instructions Communes** **5**

II.1 Structure du Dépôt . . . . . 5

II.2 Qu'est-ce qu'une Preuve de Concept Valide . . . . . 5

II.3 Processus de Soumission . . . . . 6

II.4 Règle des Fichiers Supplémentaires Interdits . . . . . 7

II.5 Format du Rapport . . . . . 7

II.6 Cibles Autorisées Uniquement . . . . . 8

II.7 Citez Toutes Vos Sources . . . . . 8

**III Instructions sur l'IA** **10**

III.1 Aperçu de la Politique . . . . . 10

III.2 Utilisations Autorisées . . . . . 10

III.3 Utilisations Interdites . . . . . 11

III.4 Responsabilité Personnelle . . . . . 11

III.5 Citation Obligatoire de l'IA . . . . . 11

III.6 Évaluation par les Pairs et l'IA . . . . . 12

III.7 Rappel Éthique . . . . . 13

**IV Avant-propos** **14**

**V Aperçu du module** **15**

**VI Exercise 00** **16**

**VII Exercise 01** **17**

**VIII Exercise 02** **18**

**IX Exercise 03** **20**

**X Exercise 04** **21**

**XI Rendu et évaluation** **22**

**XII Ressources par exercice** **23**

**XIII Sources** **25**

# Chapter I

# Instructions

- Deux jours.
- Travail individuel, revue par un pair à la fin.
- Analyse statique uniquement : pas d'exploit, pas de cible de production.
- Les binaires utilisés doivent être des versions publiques et légales.

## Format du rendu — en-tête obligatoire

Tout fichier rendu dans ce module doit commencer par l'en-tête ci-dessous, **adapté à la syntaxe de commentaire du langage** utilisé.

### Modèle générique

```
/**
 * Module: <code module>
 * Exercise: <numéro exo>
 * File: <nom du fichier>
 * Author: <votre login JeuneCrack>
 *
 * Description: <une phrase, pas de roman>
 *
 * Return: <voir ci-dessous>
 */
```

### Login

Votre identifiant **JeuneCrack** (ex: `tedsig42`). Pour un rendu en binôme : `login1/login2`. Ajoutez un handle Matrix après le login si vous le souhaitez (ex: `tedsig42 @tedsig42`).

### Return — selon le type de fichier

| Type de fichier | Valeur de Return |
|---|---|
| C (`.c`) | `return (0);` (succès), `return (-1);` (erreur), `return (valeur);` (résultat calculé) |
| Python (`.py`) | `0` (succès), `1` (erreur), `N/A` (analyse, script sans exit) |
| Shell (`.sh`) | `exit 0` (succès), `exit 1` (erreur) |
| Markdown (`.md`) | `N/A` — les fichiers d'analyse n'ont pas de valeur de retour |
| Exploit C/Python | `shell` (shell obtenu), `flag: <flag>` (flag capturé), `proof: <hash>` (preuve de concept) |
| Binaire (`.bin`) | `N/A` — pas d'en-tête pour les fichiers binaires |

### Exemples par langage

**C (`.c`) :**

```c
/**
 * Module: F-EXP-001
 * Exercise: ex02
 * File: exploit.c
 * Author: tedsig42
 *
 * Description: Buffer overflow with ret2libc
 *
 * Return: 0
 */
```

**Python (`.py`) :**

```python
# Module: F-WEB-001
# Exercise: ex05
# File: ssrf-exploit.py
# Author: tedsig42
#
# Description: Blind SSRF bypass via redirect
#
# Return: 0
```

**Shell (`.sh`) :**

```shell
# Module: F-SUP-001
# Exercise: ex03
# File: dependency-confusion.sh
# Author: tedsig42
#
# Description: Automated dependency confusion test
#
# Return: exit 0
```

**Markdown (`.md`)** — analyse, rapport :

```
<!--
Module: M-SRC-001
Exercise: ex04
File: audit-report.md
Author: tedsig42

Description: Code audit of vulnerable-httpd

Return: N/A
-->
```

# Chapter II

# Instructions Communes

## II.1 Structure du Dépôt

Votre dépôt Git doit respecter la convention de nommage suivante pour tous les modules soumis. L'outil d'évaluation (`pendragon`) se base sur cette structure pour localiser et tester votre travail.

```
<login>/
+-- <MODULE-ID>-<slug>/         # ex. M-BLS-001-business-logic/
|   +-- <turnindir>/            # ex. ex00-pricing-manipulation/
|   |   +-- <file1>             # ex. poc-pricing.py
|   |   +-- <file2>             # ex. report.md
|   |   +-- ...
|   +-- <turnindir>/            # ex. ex01-race-conditions/
|   |   +-- ...
|   +-- ...
+-- ...
```

- **Répertoire du module** : le nom complet du répertoire tel qu'affiché dans votre curriculum (ex. `M-BLS-001-business-logic`).
- **Répertoire de rendu** : la valeur de `\turnindir` fournie dans chaque exercice (ex. `ex00-pricing-manipulation`).
- **Fichiers de l'exercice** : exactement les fichiers listés dans `\exfiles`. Ni fichiers supplémentaires, ni fichiers manquants.

## II.2 Qu'est-ce qu'une Preuve de Concept Valide

Une PoC doit être :

1. **Reproductible**. N'importe qui avec le même environnement peut l'exécuter et observer la vulnérabilité. Les chemins codés en dur, les adresses IP absolues et les jetons expirés ne sont pas acceptables.

2. **Documentée**. Chaque étape est expliquée. L'évaluteur ne devrait pas avoir à deviner ce que fait le script.

3. **Ciblée**. L'exercice spécifie une cible autorisée (lab, maquette, binaire local ou plateforme désignée). Ne testez jamais les PoC sur des systèmes de production qui ne vous appartiennent pas.

4. **Minimale**. La PoC démontre *uniquement* la vulnérabilité en question. Les charges utiles non liées et le bruit sont pénalisés.

① Une PoC qui modifie silencieusement l'état du système, supprime des fichiers ou persiste des changements en dehors du périmètre de l'exercice sera rejetée. Si votre exploit nécessite un nettoyage, documentez les étapes de nettoyage dans votre rapport.

## II.3 Processus de Soumission

1. Réalisez les livrables de l'exercice sur votre machine locale.

2. Committez et poussez vers votre dépôt Git assigné.

3. Le dépôt est gelé après la date limite.

4. Les pairs clonent votre dépôt et évaluent en utilisant :

   - `pendragon` pour l'exécution automatisée des tests (là où le module fournit des configurations mouli).
   - La grille manuelle pour la qualité du rapport, le style de code et la méthodologie.



Exécutez toujours `pendragon run <MODULE-ID> <ex_id> <path/to/student/repo>` avant votre commit final pour vérifier que votre soumission passe les vérifications automatisées.

### Test avec pendragon

Si le module inclut une configuration mouli (fichiers JSON dans `resources/mouli/`), vous pouvez tester votre soumission localement :

```
pendragon run M-BLS-001 ex00 ~/student-repos/tedsig42
```

`pendragon` résout le chemin de l'exercice automatiquement en cherchant `<mod-fullname>/<ex-dirname>/` dans le répertoire étudiant donné.

### Vérification de style avec caliban

Pour les modules qui nécessitent du code C, `caliban` vérifie la conformité du style :

```
caliban check ~/student-repos/tedsig42/M-BLS-001-business-logic
```

`caliban` applique la Norme 42 (taille des fonctions, nommage des variables, constructions interdites) adaptée aux contextes de recherche de vulnérabilités. Exécutez-le avant la soumission — un échec aux vérifications de style peut invalider l'exercice.

## II.4 Règle des Fichiers Supplémentaires Interdits

- Votre dépôt doit contenir **uniquement** les fichiers listés dans `\exfiles` pour chaque exercice.
- Fichiers interdits : `.DS_Store`, `__pycache__`, `.venv/`, `node_modules/`, `.git/`, binaires compilés (sauf explicitement listés dans `exfiles`), fichiers d'échange d'éditeur, fichiers journaux et tout fichier non demandé.
- Utilisez `.gitignore` pour exclure automatiquement les fichiers indésirables.

## II.5 Format du Rapport

Lorsqu'un rapport (`report.md`) est requis, il doit inclure :

1. **Classe de vulnérabilité** (identifiant CWE si applicable).
2. **Étapes de reproduction** — numérotées, prêtes à être copiées-collées.
3. **Charges utiles** — chaînes exactes, requêtes ou entrées utilisées.
4. **Impact** — ce qu'un attaquant peut accomplir.
5. **Mitigation** — comment le développeur devrait corriger.
6. **Sources** — toutes les ressources consultées (voir Section II.7).

### En-tête de Rendu

Chaque fichier source doit inclure l'en-tête de rendu défini dans `../../../resources/latex/deliverable-header.tex`, adapté à la syntaxe de commentaire du langage utilisé.

## II.6 Cibles Autorisées Uniquement

⚠ Vous pouvez **uniquement** tester les cibles explicitement listées dans l'exercice (labs, serveurs locaux, maquettes, binaires désignés). Tester des systèmes de production réels, des applications tierces ou toute cible qui ne vous appartient pas ou pour laquelle vous n'avez pas d'autorisation écrite est interdit et peut entraîner une disqualification immédiate.

L'éthique du bug bounty s'applique tout au long de ce curriculum :

- Aucun test sur des programmes en production sans un accord de périmètre valide.
- Aucun déni de service sauf si l'exercice l'exige explicitement.
- Aucune exfiltration de données au-delà de ce qui est nécessaire pour démontrer la vulnérabilité.
- Si vous accédez accidentellement à des données réelles d'utilisateurs, arrêtez immédiatement et documentez ce qui s'est passé.

## II.7 Citez Toutes Vos Sources

Toute ressource consultée pendant votre travail doit être citée. Cela inclut :

- Articles académiques et billets de blog.
- Rapports publics de bug bounty (HackerOne, Bugcrowd, etc.).
- Documentation officielle (RFC, pages de manuel, documentation fournisseur).
- **Outils d'IA** : nom du modèle, résumé de la question, date et méthode de vérification (voir le chapitre Instructions IA pour le format de citation exact).
- Extraits de code adaptés de projets open-source (avec attribution de licence si requis).

### Format de citation

```
# R\'ef\'erences

- [1] PortSwigger, "Business Logic Flaws",
  https://portswigger.net/web-security/logic-flaws
  (consult\'e le 2025-03-15).
- [2] Rapport HackerOne #123456, "IDOR on /api/users",
  anonymis\'e, consult\'e via divulgation publique.
- [3] G\'en\'er\'e avec l'assistance de Claude 3.5 Sonnet
  le 2025-03-14. Question : "Write a Python script that...".
  V\'erifi\'e par : revue manuelle de chaque requ\^ete HTTP.
```

## Attribution en ligne

Si un exercice vous demande explicitement d'utiliser une technique ou un outil spécifique, vous n'avez pas besoin de citer l'exercice lui-même comme source. Citez tout ce qui se trouve en dehors de l'énoncé de l'exercice.

# Chapter III

# Instructions sur l'IA

## III.1 Aperçu de la Politique

Les outils de codage assistés par IA (ChatGPT, Claude, Copilot, Gemini, Cursor, Codeium, et similaires) sont **autorisés** tout au long de ce cursus. La recherche en sécurité est un domaine où la maîtrise de l'IA devient essentielle — apprendre à utiliser ces outils efficacement fait partie de votre formation.

Cependant, l'objectif est la **compréhension**, pas l'automatisation. Si vous ne pouvez pas expliquer chaque ligne de votre exploit sans l'aide de l'IA, vous n'avez pas terminé l'exercice.

① Utiliser l'IA pour contourner l'objectif pédagogique d'un exercice — par exemple, lui demander de rétro-ingéniérer un binaire pour ne pas avoir à apprendre le désassembleur, ou de générer une chaîne d'exploitation complète que vous ne comprenez pas — va à l'encontre du but du cursus et sera traité comme une fraude académique.

## III.2 Utilisations Autorisées

- **Explication de concepts**. Demander à l'IA de clarifier une classe de vulnérabilité, un sous-système du noyau, ou un protocole réseau.
- **Échafaudage de code**. Générer du code de départ pour un harnais, un fuzzer, ou un squelette de PoC — puis le modifier et l'étendre vous-même.
- **Débogage**. Expliquer des messages d'erreur, la sortie d'un désassembleur, ou un comportement de crash inattendu.
- **Résumé de documentation**. Condenser une longue RFC ou un article technique en notes exploitables.
- **Traduction**. Traduire du contenu technique entre l'anglais et le français (les deux langues sont utilisées dans ce cursus).

- **Rédaction de rapports**. Demander de l'aide pour structurer un rapport de bug bounty — mais remplir le contenu technique vous-même.

## III.3 Utilisations Interdites

- **Génération complète d'exploit**. Demander à l'IA d'écrire l'exploit complet d'un exercice, puis le soumettre sans le comprendre.
- **Contournement des contraintes de l'exercice**. Utiliser l'IA pour contourner une restriction que l'exercice impose explicitement (p. ex. « n'utilisez pas de scanners automatisés »).
- **Découverte automatisée de vulnérabilités**. Exécuter des outils de scan pilotés par IA qui remplacent l'analyse manuelle que vous êtes censé pratiquer.
- **Rédaction des grilles d'évaluation**. Vous n'êtes pas autorisé à utiliser l'IA pour générer des critères d'évaluation par les pairs ou des réponses.
- **Plagiat sans attribution**. Copier le résultat de l'IA sans le citer (voir Section III.5).

## III.4 Responsabilité Personnelle

Lorsque vous demandez à l'IA de générer du code d'exploit, vous devenez responsable de chaque octet de ce résultat avant de l'utiliser contre une cible.

1. **Lisez le code généré** ligne par ligne.
2. **Identifiez toutes les hypothèses** faites par l'IA (architecture, version du noyau, drapeaux du compilateur, topologie réseau).
3. **Testez en isolation** avant d'exécuter contre une cible d'exercice.
4. **Comprenez les modes d'échec**. Si l'exploit généré plante ou se bloque, vous devez diagnostiquer et corriger le problème vous-même.



Si l'IA génère du code que vous ne comprenez pas, demandez-lui de l'expliquer. Si vous ne comprenez toujours pas après l'explication, le code n'est pas prêt à être utilisé. Passez à une approche plus simple que vous maîtrisez entièrement.

## III.5 Citation Obligatoire de l'IA

Chaque fichier contenant du code généré ou assisté par l'IA doit inclure un commentaire de citation. Le format est :

```
# Generated with assistance from <tool> on <date>.
# Prompt: <one-line summary of the prompt>
# Verified by: <method used to verify the output>
```

### Exemples

**Python** :

```
# Generated with assistance from Claude 3.5 Sonnet on 2025-03-14.
# Prompt: "Write a Python script that sends HTTP requests with
# modified price fields to /api/checkout/pay"
# Verified by: manual review of each request and response
```

**C** :

```
/* Generated with assistance from GitHub Copilot on 2025-03-14.
 * Prompt: "Implement a stack-based buffer overflow with
 * ret2libc for x86_64 Linux"
 * Verified by: compiled with -fno-stack-protector and
 * tested against local binary with ASLR disabled */
```

**Markdown (rapport)** :

```
<!--
Generated with assistance from ChatGPT-4 on 2025-03-14.
Prompt: "Structure a bug bounty report for IDOR with
sections: summary, steps, impact, mitigation"
Verified by: all technical content written manually,
AI only suggested structure
-->
```

### Interactions multiples avec l'IA

Si vous avez utilisé plusieurs outils d'IA, ou itéré à travers plusieurs invites, citez chaque interaction significative. Un groupe d'invites connexes peut être résumé :

```
<!--
AI assistance used in this file:
- Claude 3.5 Sonnet (2025-03-14): harness skeleton generation
- ChatGPT-4 (2025-03-14): crash triage help
- Copilot (2025-03-15): inline autocomplete for loop logic
All generated code reviewed and modified manually.
-->
```

## III.6 Évaluation par les Pairs et l'IA

Lors de l'évaluation par les pairs, l'évaluateur peut :

- Vous demander d'**expliquer tout code généré par l'IA** ligne par ligne.

- Vous demander de **reproduire le raisonnement** sans l'aide de l'IA.
- Demander à voir votre **historique d'invites** pour un exercice donné.
- Déduire des points si la citation de l'IA est manquante ou si vous ne pouvez pas expliquer le code que l'IA a généré.

① Présenter le code généré par l'IA comme votre propre travail (défaut de citation) est du plagiat. Si l'évaluateur soupçonne du plagiat, il peut saisir l'équipe pédagogique pour examen.

## III.7 Rappel Éthique

- Les modèles d'IA peuvent produire du code incorrect, dangereux, ou illégal. Vous êtes **responsable** de vérifier chaque résultat avant exécution.
- Les modèles d'IA entraînés sur la recherche publique en sécurité peuvent reproduire textuellement du code provenant de projets sous licence GPL. Vérifiez les licences avant d'intégrer ces éléments dans votre travail.
- Ne saisissez pas de code sensible ou propriétaire dans des outils d'IA hébergés dans le cloud. Utilisez des modèles locaux (Llama, Mistral, DeepSeek) pour les travaux impliquant des cibles non publiques.
- L'éthique de la recherche en sécurité que vous apprenez ici s'applique à l'utilisation de l'IA : ne demandez pas à l'IA de générer des malwares, des rançongiciels, ou du code destiné à nuire à des systèmes en dehors du cadre de l'exercice.



La maîtrise des outils d'IA est une compétence recherchée dans la chasse aux bogues moderne. Utilisez-les sagement, citez-les honnêtement, et conservez toujours votre propre compréhension de chaque exploit que vous déployez.

# Chapter IV

# Avant-propos

Un patch diff est une fenêtre sur la root cause. Le message de commit dit ce que le développeur *croit* avoir corrigé ; le diff montre ce qu'il a *effectivement* changé ; le binaire montre ce que le compilateur a *réellement* produit.

Les trois récits ne coïncident pas toujours. Votre travail est de les confronter, pas de choisir le plus confortable.



L

e patch diffing ne remplace pas la lecture du code. Il sert à repérer les changements de contrôle, les constantes, les bornes et les appels qui n'apparaissent pas dans le message de commit.

# Chapter V

# Aperçu du module

| Ex | Titre | Difficulté | Points | |
|---|---|---|---|---|
| 00 | Archéologie de commit | Facile | 3 | |
| 01 | Du diff à la root cause | Moyen | 5 | |
| 02 | Diff binaire | Difficile | 5 | |
| 03 | Mesurer le patch gap | Moyen | 4 | |
| 04 | Écrire un finding | Difficile | 5 | |
| | **Total** | | **22** | |

# Chapter VI

# Exercise 00

|  | Exercise 00 |
|---|---|
| Archéologie de commit | |
| Turn-in directory : `ex00/` | |
| Files to turn in : `commit-archaeology.md` | |
| Allowed functions : `git, grep, un éditeur de texte` | |

Choisissez CVE-2023-4244 et CVE-2024-1085. Retrouvez leurs commits de correction et replacez-les dans l'historique du fichier.

## Mission

Dans `commit-archaeology.md` :

1. listez les commits, leurs dates, leurs auteurs et leurs messages ;
2. identifiez les commits qui ont introduit ou retiré la garde concernée ;
3. construisez une chronologie de trois lignes ;
4. expliquez pourquoi le second correctif n'est pas une simple duplication du premier.

```
git log --oneline --decorate -- net/netfilter/nf_tables_api.c | head -40
git show --stat <commit>
git blame -L <debut>,<fin> net/netfilter/nf_tables_api.c
```

# Chapter VII

# Exercise 01

|  | Exercise 01 |
|---|---|
| Du diff à la root cause | |
| Turn-in directory : `ex01/` | |
| Files to turn in : `root-cause.md` | |
| Allowed functions : `git, grep, un éditeur de texte` | |

Prenez CVE-2023-35001. Le correctif modifie la manière dont un champ de registre est lu dans `nft_byteorder_eval()`.

## Mission

Dans `root-cause.md` :

1. décrivez la structure de données et la disposition mémoire des champs concernés ;
2. montrez comment une valeur d'entrée atteint l'accès hors limites ;
3. distinguez la *root cause* de la *condition d'exploitation* ;
4. expliquez pourquoi le correctif ne protège pas les autres opérations qui manipulent des registres ;
5. proposez une requête `grep` qui trouve les opérations analogues.



La root cause est souvent une hypothèse de type ou de taille. La condition d'exploitation est souvent un chemin d'appel et un état de registre. Ne mélangez pas les deux.

# Chapter VIII

# Exercise 02

|  | Exercise 02 |
|---|---|
| Diff binaire | |
| Turn-in directory : `ex02/` | |
| Files to turn in : `bindiff.md` | |
| Allowed functions : `objdump, radiff2, Diaphora ou BinDiff, un éditeur de texte` | |

Comparez deux versions publiques d'un binaire contenant la fonction corrigée. Si vous n'avez pas Diaphora ou BinDiff, utilisez `objdump -d` et `radiff2`.

## Mission

Dans `bindiff.md` :

1. identifiez la fonction dans les deux binaires ;
2. comparez les blocs de base et les sauts conditionnels ;
3. repérez la constante, la borne ou l'appel ajouté ;
4. expliquez ce que le diff binaire montre que le diff source ne montre pas ;
5. listez les limites de votre analyse (optimisation, inlining, stripping).



N

e diffusez pas de binaire propriétaire. Utilisez uniquement des binaires que vous avez le droit d'analyser.

# Chapter IX

# Exercise 03

|  | Exercise 03 |
|---|---|
| Mesurer le patch gap | |
| Turn-in directory : `ex03/` | |
| Files to turn in : `patch-gap.md patch-gap.py` | |
| Allowed functions : `git, python3, un éditeur de texte` | |

Choisissez cinq CVE du papier. Pour chacune, mesurez :

- la date du commit amont ;
- la date d'arrivée dans la branche stable ;
- la date de publication de l'avis ;
- la date de disponibilité du correctif dans une distribution.

Écrivez `patch-gap.py` qui prend un fichier CSV de commits et produit un tableau avec les écarts en jours. Le script doit gérer les fuseaux horaires et les dates manquantes.

Dans `patch-gap.md`, distinguez :

1. le temps de correction (commit amont) ;
2. le temps de backport (branche stable) ;
3. le temps d'activation (redémarrage, bascule de flag) ;
4. le temps d'exposition (systèmes non patchés).

Classez ensuite les mécanismes d'urgence décrits par Project Zero (feature flags, filtering, dual stack) et expliquez pourquoi ils réduisent l'exposition sans fermer la root cause.

# Chapter X

# Exercise 04

|  | Exercise 04 |
|---|---|
| Écrire un finding | |
| Turn-in directory : `ex04/` | |
| Files to turn in : `finding.md` | |
| Allowed functions : `un éditeur de texte` | |

Rédigez un finding de deux pages maximum sur le candidat le plus solide de vos exercices précédents. Le finding doit être compréhensible par un ingénieur sécurité qui ne connaît pas le sous-système.

Structure imposée :

1. **Titre** : composant, fonction, type de faiblesse.
2. **Résumé** : trois phrases.
3. **Contexte** : version, commit, configuration.
4. **Root cause** : mécanisme précis.
5. **Preuve** : extraits de code, commandes, diff.
6. **Impact** : ce qui est atteignable, et ce qui ne l'est pas.
7. **Remédiation** : correctif proposé ou question à poser.
8. **Atténuation d'urgence** : ce qui réduit l'exposition sans fermer la root cause.
9. **Limites** : ce que vous n'avez pas vérifié.

Un finding n'est pas un article de blog : pas de storytelling, pas de conclusion triomphale. Des faits, des preuves, des limites.

# Chapter XI

# Rendu et évaluation

```
A-EPI-001-patch-diffing/
|-- ex00/commit-archaeology.md
|-- ex01/root-cause.md
|-- ex02/bindiff.md
|-- ex03/
|   |-- patch-gap.md
|   +-- patch-gap.py
+-- ex04/finding.md
```

| **Critère** | **Attendu** | **Points** |
|---|---|---|
| Archéologie | Chronologie et commits corrects | 4 |
| Root cause | Mécanisme distingué de l'exploitation | 5 |
| Diff binaire | Changement de contrôle identifié | 4 |
| Patch gap | Mesure reproductible et limites | 4 |
| Finding | Preuves, impact et limites clairs | 5 |

# Chapter XII

# Ressources par exercice

### Exercice 00 — Archéologie de commit

- Noyau Linux : https://git.kernel.org/
- Papier, sections 4 et 8.
- OtterSec, OtterRoot : https://osec.io/blog/netfilter-universal-root-1-day/
- À chercher : `git log -follow`, `git blame`, `Fixes:`, `introduced by`.

### Exercice 01 — Du diff à la root cause

- NVD : CVE-2023-35001, CVE-2023-4244, CVE-2024-1085, CVE-2024-26809.
- OtterSec, OtterRoot : https://osec.io/blog/netfilter-universal-root-1-day/
- Papier, section 6.
- À chercher : `nft_byteorder`, `nft_is_active_next`, `nft_pipapo_destroy`, `set->dead`.

### Exercice 02 — Diff binaire

- BinDiff : https://www.zynamics.com/bindiff.html
- Diaphora : https://github.com/joxeankoret/diaphora
- Ghidra : https://ghidra-sre.org/
- Papier, section 5.
- À chercher : `function matching`, `basic block`, `control-flow graph`.

### Exercice 03 — Mesurer le patch gap

- Project Zero, emergency patching : https://projectzero.google/2026/10/emergency-patching.html

- OtterSec, patch gap mobile : https://osec.io/blog/patch-gap-to-mobile-renderer-rce/
- NVD, CVE-2024-26809 : https://nvd.nist.gov/vuln/detail/CVE-2024-26809
- À chercher : `commit date`, `stable tag`, `distribution backport`, `activation date`.

## Exercice 04 — Écrire un finding

- Papier, section 8.
- NVD : https://nvd.nist.gov/
- Dépôt compagnon : `validation/methodology.md`.
- À chercher : `root cause`, `affected versions`, `proof of concept`, `limitations`.

# Chapter XIII

# Sources


- Phrack, *Security is failure: Every Patch Is a Guess*, sections 4, 5 et 8.
- NVD : CVE-2023-35001, CVE-2023-4244, CVE-2024-1085, CVE-2024-26809.
- Project Zero, *How to fix a bug in a fix* : https://projectzero.google/2026/10/emergency-patching.html
- OtterSec, *OtterRoot: Netfilter universal root 1-day* : https://osec.io/blog/netfilter-universal-root-1-day/
- OtterSec, *Patch gap to mobile renderer RCE* : https://osec.io/blog/patch-gap-to-mobile-renderer-rce/
- `pwning.tech/nftables`.
- Documentation BinDiff et Diaphora.